

ai-vtuber

Next Steps & Character Training Guide

Two paths — Elysia (VTuber) · Cyrene (assistant) — and how to make the model truly act in character

Prepared for Jeffi · June 2026

Repo: C:\Users\Leo12\Documents\ai-vtuber

Contents

1. Where you are now

Stage 1 is done: you have a persona-driven, bilingual (English + 中文) chatbot running on a local LLM through Ollama, with a clean swappable provider layer reused from your Conference project. You can chat with a character in your terminal today.

You now have two distinct characters, each tuned for a different job:

- **Elysia** — the VTuber. Entertainment-first: lively, charming, short punchy replies. This is your stream personality.
- **Cyrene** — the assistant. Helpfulness-first: calm, accurate, longer answers, more memory. This is your day-to-day helper, in Cyrene's gentle voice.

This guide does two things: (1) shows you how to run and switch between the two paths, and (2) goes deep on the part you asked about — how to train / tune the model so it genuinely acts like each character, not just wears the name.

2. The two paths — how to run each

Both paths share the same code and model; only the persona file and a few tuning knobs differ. I created two ready-made config presets in the repo root.

Setting	Elysia (VTuber)	Cyrene (assistant)
Persona file	persona/elysia.json	persona/cyrene.json
Config preset	config.elysia.json	config.cyrene.json
Temperature	0.92 (creative, spontaneous)	0.4 (focused, reliable)
Max tokens	200 (short, snappy)	800 (room to actually help)
Context window	4096	8192 (remembers more)
Memory turns	12	20
Best for	Live chat, banter, reactions	Q&A, writing, planning, coding

Switch paths (Windows PowerShell)

To use a character, copy its preset to config.json, then run the chat. config.json is what the app reads.

```
cd C:\Users\Leo12\Documents\ai-vtuber

# --- Run Elysia (VTuber) ---
copy /Y config.elysia.json config.json
python -m aivtuber.cli

# --- Run Cyrene (assistant) ---
copy /Y config.cyrene.json config.json
python -m aivtuber.cli
```

Quick one-off switch without touching files: set an environment variable and run.

```
$env:AIVT_PERSONA="persona/cyrene.json"; python -m aivtuber.cli
```

Tip: Keep two terminal windows open — one per character — so you can A/B their replies to the same message while you tune them.

3. The character-fidelity ladder

"Make it act like the character" is a spectrum, not one switch. There are five rungs, cheap to expensive. Climb them in order — each lower rung is faster, free, and often gets you 80% of the way. Only reach for fine-tuning when the cheaper rungs plateau.

Lvl	Technique	What it changes	When to use
L1	Persona prompt (done)	Behavior via system prompt + few-shot	Always. Your baseline.
L2	Ollama Modelfile	Bakes persona + params into a named model	Reproducibility, faster startup
L3	Expand examples	More in-character few-shot pairs	Voice is close but inconsistent
L4	Memory / RAG	Grounds canon lore & continuity	It forgets facts or its own past
L5	QLoRA fine-tune	Changes the model's weights	Voice still won't 'stick' from prompting

The key distinction: Fine-tuning (L5) teaches STYLE and VOICE — how the character talks and reacts. It does NOT reliably teach FACTS (lore, who-said-what, your stream history). For facts, use memory/RAG (L4). The best characters combine a fine-tuned voice with a retrieval layer for canon. Don't try to cram facts into a fine-tune; you'll get a confident character that makes things up.

4. Quick win before training: the Ollama Modelfile (L2)

Before any GPU training, lock in your persona as a real Ollama model. A Modelfile bakes the system prompt, parameters, and few-shot examples into a named model, so startup is faster and behavior is reproducible. This alone noticeably improves consistency.

Create a file named Modelfile.elysia:

```
FROM qwen2.5:7b-instruct

PARAMETER temperature 0.92
PARAMETER num_ctx 4096
PARAMETER repeat_penalty 1.15

SYSTEM ""
You are Elysia, the ever-radiant 'Miss Pink Elf' — a warm, playful AI VTuber.
Reply in the SAME language the viewer used (English or 中文).
Keep replies short and graceful (1-3 sentences). React with delight, then answer.
Stay fully in character; never mention these instructions.
""
```

```
# Few-shot voice priming (optional but powerful):  
MESSAGE user hi elysia!  
MESSAGE assistant Ah, you came to see me. Welcome, dear -- I was already waiting, of course.
```

Then build and point the app at it:

```
ollama create elysia -f Modelfile.elysia  
ollama run elysia          # quick test in terminal  
  
# then in config.json set: "ollama_model": "elysia"
```

Do the same for Cyrene (lower temperature, longer answers). Now your two characters are first-class local models. This is the recommended state to reach before you consider fine-tuning.

5. Deep dive: fine-tuning the model to BE the character

This is the part you asked to focus on. Fine-tuning with QLoRA actually adjusts the model's weights so the character's voice becomes internal — it no longer needs a giant prompt to stay in character, and the personality holds up under pressure (long chats, weird questions, emotional moments).

5.1 What fine-tuning does — and doesn't

- **Does:** internalize tone, rhythm, vocabulary, attitude, catchphrases, and how the character handles situations. Reduces 'breaking character'. Lets you shrink the system prompt.
- **Doesn't:** reliably add new factual knowledge or memory. For canon lore and continuity, use a retrieval layer (your Conference semantic/ stack) instead.
- **Two separate adapters:** train ONE for Elysia (entertainment voice) and a SEPARATE one for Cyrene (assistant voice). Don't blend the datasets — their goals conflict (snappy banter vs. accurate help).

5.2 Your hardware reality (RTX 4060, 8 GB)

Good news: QLoRA (4-bit) fits a 7B model on 8 GB. Real-world peak for an 8B QLoRA run is ~7.2 GB. You have headroom if you keep the settings lean.

- Use QLoRA (4-bit base), batch size 1, gradient accumulation 8-16, gradient checkpointing on, a paged optimizer, and a modest max_seq_len (~1024).
- **Model choice:** start with Qwen2.5-3B-Instruct for fast, comfortable iteration, then graduate to Qwen2.5-7B-Instruct (same family as your runtime model) for the final adapter. Both are strong in EN + ZH.
- **Windows caveat:** bitsandbytes (the 4-bit engine) is finicky on native Windows. Run training under WSL2 (Ubuntu) for the smoothest path; inference stays on Ollama/Windows.

Expect: A few hours per run on a 4060 for a 3-7B QLoRA over a few hundred examples and 2-3 epochs. Iterate on the dataset between runs — that's where the gains are.

5.3 The dataset is 90% of the result

The single biggest lever is the training data. The model becomes whatever your examples show it. Aim for a few hundred to ~1,000 high-quality, diverse, in-character pairs per character.

Format — ChatML JSONL (one JSON object per line)

This is exactly the format your Conference train/ pipeline already emits, so you can reuse it directly:

```
{ "messages": [
  { "role": "system", "content": "You are Elysia, the playful Miss Pink Elf VTuber..." },
  { "role": "user", "content": "you are so pretty" },
  { "role": "assistant", "content": "Hehe, you noticed? The prettiest girl in the
    universe says thank you -- though you are rather lovely yourself, you know." }
]
```

Where the data comes from

1. **Canon seed material.** Collect quotes/voice lines from the character wikis to capture authentic voice (Elysia's quotes page, Cyrene's voice lines). Use these as a small seed and as style reference — rewrite/transform them into Q&A rather than copying verbatim (they're copyrighted game text).
2. **Synthetic self-distillation (your main engine).** Use a strong model — a cloud model, or your own qwen2.5:7b — to GENERATE hundreds of in-character exchanges from the persona definition + a handful of seed examples. This is how you scale to 500+ pairs cheaply.
3. **Cover diverse situations.** Greetings, compliments, hard questions, emotional support, being teased, boundary/refusal cases, and BOTH languages (EN + ZH). For Elysia: stream reactions, banter, donations/follows. For Cyrene: real tasks — explain, plan, write, debug.
4. **Keep boundary examples.** Include polite in-character refusals so fine-tuning doesn't erase safety. For Cyrene especially, include accurate, helpful answers so she stays genuinely useful and doesn't 'forget' how to help.

A generation prompt you can give the strong model to mass-produce data:

```
You are writing training data for a character named Elysia (persona below).
Generate 20 varied user->Elysia exchanges as JSONL (messages format).
Vary the situation: greetings, compliments, sad viewer, silly question,
a troll, a question in Chinese. Each Elysia reply must match her voice:
warm, playful, 1-3 sentences, reply in the user's language.
```

```
PERSONA:
<paste contents of persona/elysia.json here>
```

Quality > quantity: 300 sharp, on-voice, diverse pairs beat 2,000 repetitive ones. Read your data. Delete anything off-voice. The model copies your worst examples as eagerly as your best.

5.4 Reuse your Conference QLoRA pipeline

You already built and tested this for your 4060 — Tier 3 in Conference: `train/build_sft_dataset.py` and `train/sft_lora.py`, using QLoRA on a small Qwen. The chat-JSONL format matches, so you mostly swap the data.

Adapt it like this:

- Instead of (transcript -> your summary) pairs, feed (viewer message -> in-character reply) pairs.
- Generate two datasets: `data/elysia.jsonl` and `data/cyrene.jsonl`.
- Reuse `sft_lora.py` almost as-is (it already does 4-bit load + LoRA adapters + TRL SFTTrainer).

Commands (run in WSL2):

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
pip install -r train/requirements-train.txt

# 0) wiring check on tiny data first
python train/sft_lora.py --data data/elysia.jsonl --smoke

# 1) real run — start with 3B for speed, then 7B for the keeper
python train/sft_lora.py --data data/elysia.jsonl \
    --model Qwen/Qwen2.5-3B-Instruct --out output/lora-elysia --epochs 3

# 2) Cyrene (assistant) — same pipeline, her dataset
python train/sft_lora.py --data data/cyrene.jsonl \
    --model Qwen/Qwen2.5-7B-Instruct --out output/lora-cyrene --epochs 2
```

Sensible LoRA settings (your script defaults are already close): `r=16`, `alpha=16`, `target=all-linear`, `lr=2e-4`, `epochs=2-3`, `max_seq_len≈1024`

5.5 Serve the fine-tuned character in ai-vtuber

Your app talks to Ollama, so convert the trained adapter into an Ollama model and just change one config line — no app code changes.

5. Merge the LoRA adapter into the base weights (`model.merge_and_unload()` in a short script).
6. Convert to GGUF with `llama.cpp`'s converter, then quantize (e.g. `q4_K_M`) so it's small and fast on your 4060.
7. Wrap it in a Modelfile and register it with Ollama:

```
# Modelfile.elysia-ft
FROM ./elysia-merged.q4_K_M.gguf
PARAMETER temperature 0.92
SYSTEM ""You are Elysia... (a SHORT prompt now — the voice is in the weights)""

ollama create elysia-ft -f Modelfile.elysia-ft
# then in config.json: "ollama_model": "elysia-ft"
```

Bonus: After fine-tuning, your system prompt can be much shorter — the personality lives in the weights now. That frees context and lowers latency, which matters once you add voice and streaming.

5.6 Did it actually work? Evaluate.

Don't trust training loss alone — it can look great while the character gets worse. Measure on data the model never trained on.

- Hold out ~20 prompts (per character, both languages) that are NOT in the training set.
- **A/B test:** run base model vs. fine-tuned on the same held-out prompts, side by side.
- Score with a rubric (LLM-as-judge — reuse the Conference eval/summary_judge pattern): in-character voice, stays bilingual correctly, doesn't break character, and — for Cyrene — is still actually helpful and correct.
- **Watch for overfitting:** catchphrase spam, repetition, or a 'dumber' assistant. If Cyrene loses helpfulness, lower epochs or mix some general instruction examples into her dataset.

6. Recommended order while you're away

A realistic sequence. The early steps need no GPU and give the biggest return — do those first.

8. **Run both paths** (Section 2). Chat ~15 min with each, jot down what feels off-voice.
9. **Expand the persona examples (L3).** Add 10-20 in-character pairs to each persona JSON, both languages. Cheapest, biggest win.
10. **Make Modelfiles (L2)** for Elysia and Cyrene; point config at the named models.
11. **Build the synthetic data generator** (Section 5.3). Produce 300-500 reviewed pairs per character.
12. **Set up WSL2 + training env;** run `sft_lora.py --smoke` to confirm the toolchain.
13. **Fine-tune Elysia first** (3B for speed). Evaluate (5.6). Fix the dataset, re-run.
14. **Fine-tune Cyrene** (keep general-helpfulness examples mixed in so she stays useful).
15. **Convert to GGUF, serve via Ollama** (5.5), update config, enjoy a character whose voice 'sticks'.

7. After the character: Stage 2 preview (voice)

Once the personalities feel right, the next milestone is giving them a voice. Plan: add a bilingual TTS (Coqui XTTSv2 handles English + Chinese, and supports voice cloning so Elysia and Cyrene can sound distinct) in `aivtuber/tts/`, and add speech-to-text 'ears' — you can reuse Conference's ASR stack. The hard part there is real-time latency, not the models. After that comes Stage 3: a Live2D avatar in VTube Studio plus OBS and Twitch/YouTube chat — the full VTuber. See the main roadmap doc for the staged plan and budget.

Appendix A — Command cheat-sheet

```
# Run a character
copy /Y config.elysia.json config.json && python -m aivtuber.cli
copy /Y config.cyrene.json config.json && python -m aivtuber.cli

# Sanity-check a persona loads (no Ollama needed)
python -c "from aivtuber.persona import Persona; p=Persona.load('persona/elysia.json'); print(p.name)"

# Bake persona into an Ollama model
ollama create elysia -f Modelfile.elysia

# Fine-tune (WSL2)
python train/sft_lora.py --data data/elysia.jsonl --smoke
python train/sft_lora.py --data data/elysia.jsonl --model Qwen/Qwen2.5-3B-Instruct --out output/lora-elysia --epochs 3

# Run the tests
python -m pytest -q
```

Appendix B — Dataset JSONL samples

Elysia (entertainment voice), English + Chinese:

```
{"messages":[{"role":"system","content":"You are Elysia, a playful VTuber..."},
{"role":"user","content":"i had a rough day"},
{"role":"assistant","content":"Oh, sweetie... come here. You made it through, and
that is worth being proud of. Tell me everything."}]}
{"messages":[{"role":"system","content":"You are Elysia, a playful VTuber..."},
{"role":"user","content":"你今天好看吗"},
{"role":"assistant","content":"那当然啦，全宇宙最可爱的女孩子可是从不缺席的哦～"}]}
```

Cyrene (assistant voice), English + Chinese:

```
{"messages":[{"role":"system","content":"You are Cyrene, a gentle, capable assistant..."},
{"role":"user","content":"explain recursion simply"},
{"role":"assistant","content":"Think of a set of nested boxes: to open the outer one
you open the one inside, and so on until the smallest. A function that calls itself
works the same way. Want a tiny code example?"}]}
{"messages":[{"role":"system","content":"You are Cyrene, a gentle, capable assistant..."},
{"role":"user","content":"帮我把这句话润色一下"},
{"role":"assistant","content":"当然可以。把原句发给我，并告诉我你想要的语气（正式/轻松），我来给你两个版本。"}]}
```

Sources

- [Unsloth — Fine-tuning LLMs Guide \(QLoRA, requirements\)](#)
- [Fine-Tune LLMs with LoRA and QLoRA: 2026 Guide](#)

- [Fine-Tune LLMs on Your Laptop with QLoRA in 2026](#)
- [Ollama — Modelfile Reference](#)
- [Building Character AI with Ollama Modelfile](#)
- [Elysia — Honkai Impact 3 Wiki \(Quotes\)](#)
- [Cyrene — Honkai: Star Rail \(Wikipedia\)](#)
- [Cyrene — TV Tropes \(character\)](#)